

GMX · SECURITY REVIEW

GMX

Incentives Contracts.

A security review of Incentives Contracts.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
GMX	Incentives Contracts	Zdravko Hristov, Ali Shehab, Vlad Zotov, Michael Lett, Ferran Celades	September 7 – September 24, 2026	October 8, 2026

§ 00

Contents.

35 pages: overview, confidence ranking, scope, 17 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 15 findings	P. 09
	Remediation Review · 2 findings	P. 31
§ 05	Disclaimer	P. 34
§ 06	About Guardian	P. 35

§ 01

Overview.

Guardian reviewed Incentives Contracts across 2 review rounds. The final report includes 3 Medium, 3 Low, 11 Informational findings.

AUDIT FIRM

Guardian

CLIENT

GMX

PROJECT

Incentives Contracts

PREPARED BY

Zdravko Hristov, Ali Shehab, Vlad Zotov, Michael Lett, Ferran Celades

AUDIT WINDOW

September 7 – September 24, 2026

FINAL REPORT

October 8, 2026

REPOSITORY

<https://github.com/gmx-io/gmx-synthetics-private>

METHODOLOGY

Smart contract · Manual · Main Review · Remediation Review

MAIN REVIEW COMMIT

84d06edb9ec2

REMEDATION REVIEW COMMIT

a0afb112f915

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer's assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	0	0	0	0	0
● High	0	0	0	0	0
● Medium	3	0	1	0	2
● Low	3	0	1	1	1
● Informational	11	0	4	0	7
All severities	17	0	6	1	10

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Incentives Contracts in <https://github.com/gmx-io/gmx-synthetics-private>. The scope includes 1,560 source lines across 6 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
MAIN REVIEW		
contracts/incentive/EsGmxIssuer.sol	141	182
contracts/incentive/IEsGmxIssuer.sol	3	6
contracts/incentive/IncentiveEventUtils.sol	174	247
contracts/incentive/IRatioVester.sol	3	6
contracts/incentive/RatioVester.sol	419	529
contracts/incentive/VesterCapZeroer.sol	38	54
REMEDIATION REVIEW		
contracts/incentive/EsGmxIssuer.sol	147	194
contracts/incentive/IEsGmxIssuer.sol	3	6
contracts/incentive/IncentiveEventUtils.sol	174	247
contracts/incentive/IRatioVester.sol	3	6
contracts/incentive/RatioVester.sol	429	548
contracts/incentive/VesterCapZeroer.sol	26	39

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in. Status is where each finding ended up; its own page says how.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Backdated tranche merges shorten esGMX and pair-token lock time	Gaming	MEDIUM	RESOLVED	p. 9
M-02	Pending V1 rewards can escape cap zeroing	Logical Error	MEDIUM	RESOLVED	p. 11
M-03	Late prior-epoch fee finalization blocks the next cross-chain distribution epoch	Out Of Scope	MEDIUM	ACKNOWLEDGED	p. 13
L-01	reducedIssuedAmount clawback is evadable by claiming or migrating the issued cap before the correction lands	Logical Error	LOW	PARTIALLY RESOLVED	p. 15
L-02	Any address that has ever vested is permanently disqualified as a migration receiver because totalConvertedAmounts is never reset	Logical Error	LOW	ACKNOWLEDGED	p. 17
L-03	A frozen receiver passes isFreshForTransfer, so signalTransfer green-lights a migration that acceptTransfer always reverts after doing all its work	Logical Error	LOW	RESOLVED	p. 19
I-01	zeroCaps skip branch and withdrawToken emit no protocol event	Events	INFORMATIONAL	ACKNOWLEDGED	p. 21
I-02	epochs[i].totalAmount and totalIssuedAmount diverge after a reduction and the difference is undocumented	Documentation	INFORMATIONAL	RESOLVED	p. 22
I-03	Uncapped mode accepts conflicting issuer	Configuration	INFORMATIONAL	RESOLVED	p. 23
I-04	IRatioVester exposes no view for provisioningComplete or the transfer feature flag, so no complete migration pre-flight predicate can be built	Best Practices	INFORMATIONAL	RESOLVED	p. 24
I-05	signalTransfer accepts a self-transfer that transferVestingState always reverts	Out Of Scope	INFORMATIONAL	RESOLVED	p. 26
I-06	Ambiguous claimable() semantics during freeze	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 27
I-07	New cap retroactively unlocks old vesting	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 28
I-08	Fully reversible deposits permit a renewable transient lock of all vesting backing	denial of service / MEV	INFORMATIONAL	ACKNOWLEDGED	p. 29

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-09	Batch completion events do not index batchIndex	logic	INFORMATIONAL	RESOLVED	p. 30

REMEDIATION REVIEW

I-01	EsGmxIssuer header documentation omits public self-claim paths	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 31
I-02	Misleading comment in reduceIssuedAmount()	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 33

MEDIUM **M-01**

RESOLVED

Backdated tranche merges shorten esGMX and pair-token lock time

CATEGORY Gaming

LOCATION `contracts/incentive/RatioVester.sol:_deposit()` `contracts/incentive/RatioVester.sol:_settle()` `contracts/incentive/RatioVester.sol:_withdraw()` `contracts/incentive/RatioVester.sol:getPairRequirement()`

REVIEW Main Review

DESCRIPTION

Deposits made in the same weekly epoch are merged into the latest active tranche. The merge increases `totalAmount`, but keeps the old `startTime` and does not adjust `convertedAmount`.

In `_deposit`:

```

`solidity
Tranche[] storage list = tranches[_account];
uint256 epoch = currentEpoch();
bool merged = false;
if (list.length > trancheStartIndex[_account]) {
    Tranche storage last = list[list.length - 1];
    if ((last.startTime + EPOCH_OFFSET) / EPOCH_DURATION == epoch) {
        last.totalAmount = last.totalAmount + _amount;
        merged = true;
    }
}
`

```

Later, `_settle` calculates vested amount from the tranche's full `totalAmount` and the elapsed time since `startTime`:

```

`solidity
uint256 elapsed = endTime > tranche.startTime ? endTime - tranche.startTime : 0;
if (elapsed > vestingDuration) { elapsed = vestingDuration; }
uint256 target = tranche.totalAmount * elapsed / vestingDuration;
`

```

This means a large deposit made near the end of an epoch receives vesting credit from the earlier small deposit's timestamp.

The issue is repeatable because `withdraw()` closes the active tranche list by moving the cursor to the end:

```

`solidity

```

M-01 Backdated tranche merges shorten esGMX and pair-token lock time

CONTINUED

```
trancheStartIndex[_account] = tranches[_account].length;
```

```
`
```

After that, the next epoch's first deposit creates a fresh tranche anchor, and the same pattern can be repeated.

Example cycle with `vestingDuration = 365 days` and `pairRatioFactor = 5x` :

- Start of epoch: `deposit(1)` creates a tranche.
- End of the same epoch: `deposit(X)` merges into that tranche and gets the old `startTime`.
- Same block or next: `withdraw()` settles the tranche, converts about `1.917805%` of `X`, then returns the remaining esGMX and all pair collateral.
- `claim()` pays out the converted amount from `unpaidClaimAmounts``.

The single-epoch head start on one merge is an accepted design tradeoff that bounds the tranche count. The issue reported here is the repetition it enables, not that one-week grant. Because `withdraw()` resets `trancheStartIndex` to the tranche length, the next epoch's first deposit re-anchors a fresh tranche, so the cycle is unbounded: an account can realize roughly `1.917805%` of remaining headroom per week, compounding to about $1 - (1 - 0.01917805)^{52} = 63.47\%$ of the lifetime cap per year. The pair collateral is pulled and fully returned inside each cycle, so the 5x pair requirement is committed for a single transaction per week rather than for the vesting period.

The account therefore vests a meaningful fraction of its cap into GMX per year while avoiding the long-lived pair-collateral lock. The pair requirement is a cost meant to keep the backing GMX position committed for the vesting duration, and reducing that commitment to one transaction per week nullifies the economic purpose of the requirement while the depositor keeps the backing position liquid and unstakeable.

— RECOMMENDATION

Do not backdate newly added principal when merging. The simplest fix is to create a new tranche for each deposit, anchored at `block.timestamp`; if weekly merging is required for gas reasons, rebase the merged tranche so only the old amount keeps the old elapsed credit and the newly added amount starts with zero elapsed credit. A minimum holding period or per-epoch withdraw limit would also make the cycle harder to repeat.

— RESOLUTION

Status: Resolved.

MEDIUM **M-02**

RESOLVED

Pending V1 rewards can escape cap zeroing

CATEGORY	Logical Error
LOCATION	contracts/incentive/VesterCapZeroer.sol:zeroCaps() contracts/incentive/VesterCapZeroer.sol:getPositiveTerms() contracts/incentive/VesterCapZeroer.sol:isZeroed() contracts/v1/IVesterV1.sol:getMaxVestableAmount()
REVIEW	Main Review

DESCRIPTION

`VesterCapZeroer` is meant to close a user's legacy V1 vesting cap by raising `cumulativeRewardDeductions`. The issue is that `zeroCaps()` first checks the current legacy cap and skips the account if that cap is zero:

```

`solidity
if (IVester(vester).getMaxVestableAmount(account) == 0) { continue; }
`

```

The legacy V1 cap calculation does not include pending tracker rewards. It only sees rewards already materialized into `cumulativeRewards`, plus `transferredCumulativeRewards` and `bonusRewards`, then subtracts `cumulativeRewardDeductions`.

```

`solidity
uint256 maxVestableAmount = transferredCumulativeRewards[_account] + bonusRewards[_account];
if (rewardTracker != address(0)) {
maxVestableAmount += rewardTracker.cumulativeRewards(_account);
}
return maxVestableAmount > deduction ? maxVestableAmount - deduction : 0;
`

```

But `VesterCapZeroer.getPositiveTerms()` intentionally includes pending `claimable()` rewards:

```

`solidity
terms += IRewardTracker(rewardTracker).cumulativeRewards(_account)
+ IRewardTracker(rewardTracker).claimable(_account);
`

```

So an account can be in this state:

```

`text
claimable = 100
cumulativeRewards = 0
bonusRewards = 0
transferredCumulativeRewards = 0
cumulativeRewardDeductions = 0

```

M-02 Pending V1 rewards can escape cap zeroing

CONTINUED

For that account, `getMaxVestableAmount()` returns zero, so `zeroCaps()` skips and writes no deduction. `isZeroed()` also returns true, because it uses the same `getMaxVestableAmount()` check.

Later, when the account claims, compounds, stakes, or unstakes in the legacy system, the pending reward can become `cumulativeRewards`. At that point the old V1 vester starts reporting a positive vesting cap again, but `cumulativeRewardDeductions` is still zero because the zeroing step skipped the account.

The comment on `getPositiveTerms` frames the `claimable` term as safe ("only adds margin to a permanent closure"), which confirms the intent to cover unmaterialized accrual. But the `getMaxVestableAmount == 0` skip guard executes before `getPositiveTerms` is ever called, so that margin is never applied to precisely the accounts it was written for, the ones with `cumulativeRewards == 0` and `claimable > 0`. The documented intent and the code diverge exactly on the vulnerable population.

— RECOMMENDATION

Do not use `getMaxVestableAmount()` as the skip condition. Compute the intended deduction first with `getPositiveTerms(account)`, then skip only if the existing `cumulativeRewardDeductions(account)` is already greater than or equal to that value. `isZeroed()` should use the same check, so keeper verification cannot report success for an account where no deduction was written.

— RESOLUTION

Status: Resolved.

MEDIUM **M-03**

ACKNOWLEDGED

Late prior-epoch fee finalization blocks the next cross-chain distribution epoch

CATEGORY	Out Of Scope
LOCATION	packages/gmx-synthetics-private/contracts/fee/FeeDistributor.sol:distribute():L387-L419
REVIEW	Main Review

DESCRIPTION

FeeDistributor intentionally permits an authenticated response for epoch N to finish shortly after the weekly boundary, but it does not attribute that completion to N. Response-derived amounts occupy shared, non-epoch-keyed slots, and distribute records only the current wall-clock timestamp. If the permitted N response is finalized at T+1, `_validateDistributionNotCompleted` consequently treats N+1 as already distributed while the chain's published snapshot remains N. `commitSnapshot` cannot publish N+1 for the rest of the week; `initiateDistribute` cannot run from state None; and `resetDistribution` is unavailable after successful distribution because it also rejects state None. Other configured chains' N+1 reads include the stuck chain's N snapshot and fail the all-chain epoch-equality validation.

Impact: A routine cross-chain read or bridge delay across the weekly boundary on one configured chain can stall the current epoch's fee distribution across the configured network for the remainder of the week. Current GMX allocations and fee payments to stakers, keepers, treasury, and Chainlink are delayed, and allocation inputs cannot be refreshed until the next epoch. The exact-boundary control demonstrates a one-second discontinuity: N and N+1 can both be processed as their separate intended cycles at T, but the identical valid N response at T+1 consumes and blocks N+1.

RECOMMENDATION

Persist the epoch/commitment associated with the accepted read response and record completion against that epoch rather than against the current timestamp. Maintain an epoch-keyed completion guard so a permitted late epoch-N finalization does not consume epoch N+1, and provide a safe recovery transition that can clear stale working data and publish the current snapshot without weakening replay protection.

PROOF OF CONCEPT <https://github.com/GuardianOrg/SentryRepo-Incentivize-Contracts---Round-1-01a07bc4c3fa75a58f6034e9f4cbf817/blob/refinery/poc-01a08029-5151-7311-9262-a30cedc32167/test/refinery-poc/late-prior-epoch-fee-finalization-blocks-the-next-cross-chain-d94473c1.sol>

M-03 Late prior-epoch fee finalization blocks the next cross-chain distribution epoch **CONTINUED**

— **RESOLUTION**

Status: Acknowledged.

LOW **L-01**

PARTIALLY RESOLVED

reduceIssuedAmount clawback is evadable by claiming or migrating the issued cap before the correction lands

CATEGORY	Logical Error
LOCATION	contracts/incentive/EsGmxIssuer.sol:reduceIssuedAmount() contracts/incentive/EsGmxIssuer.sol:claimable() contracts/incentive/EsGmxIssuer.sol:claim() contracts/incentive/RatioVester.sol:getVestingCap() contracts/incentive/RatioVester.sol:transferVestingState()
REVIEW	Main Review

DESCRIPTION

`EsGmxIssuer.reduceIssuedAmount` is the only lever to reverse an erroneous esGMX issuance, and it is bounded by `claimable(_account) = issuedAmounts - claimedAmounts`. `RatioVester.getVestingCap` reads `issuedAmounts` gross, never net of `claimedAmounts`, so an issued amount both funds a claim and raises the vesting cap. Neither claiming nor migrating ever decrements `issuedAmounts`, so an account can drive its own `claimable` to zero with a permissionless `claim()`, or relocate the whole cap to a fresh address through `transferVestingState`, and afterwards `reduceIssuedAmount` either reverts with `IssuanceReductionExceedsUnclaimed` or targets an account whose cap has already moved, while the granted vesting capacity survives.

```

```solidity
// EsGmxIssuer.reduceIssuedAmount
uint256 unclaimedAmount = claimable(_account); // issuedAmounts - claimedAmounts
if (_amount > unclaimedAmount) { revert Errors.IssuanceReductionExceedsUnclaimed(_amount,
unclaimedAmount); }
issuedAmounts[_account] = issuedAmounts[_account] - _amount;

// getVestingCap adds issuedAmounts GROSS, so the cap outlives any claim
cap = cap + IEsGmxIssuer(issuer).issuedAmounts(_account);

// transferVestingState relocates that cap with no link back to the issuer
uint256 cap = getVestingCap(_sender);
transferredCaps[_receiver] = transferredCaps[_receiver] + cap;
transferredCapDeductions[_sender] = transferredCapDeductions[_sender] + cap;
```

```

The sequence is reachable from unprivileged calls: an account that received issuance calls `claim()`, or signals and accepts a self-transfer to a second address it controls (which

L-01 reduceIssuedAmount clawback is evadable by claiming or migrating the issued cap before the correction lands CONTINUED

runs `claimForAccount` then `transferVestingState`), and the admin's later `reduceIssuedAmount` no longer bites. Conversion itself remains 1:1 with esGMX the account has already claimed and is bounded by the `obligations <= backing` check in `_deposit` , so no GMX is created or over-drawn beyond the deposited esGMX; the residual defect is that the correction meant to shrink the account's vesting capacity is permanently escaped, leaving that capacity live to convert the erroneously issued esGMX.

— RECOMMENDATION

Consider introducing a short hold window during which freshly issued amounts cannot be claimed or migrated so a same-window `reduceIssuedAmount` still applies. Alternatively, acknowledge the finding.

In addition, you can use the minimum of the `amount` and `unclaimedAmount` during the execution of the function, so if the unclaimed amount changed only partially, the call will still succeed.

```
```diff
if (_amount > unclaimedAmount) {
- revert Errors.IssuanceReductionExceedsUnclaimed(_amount, unclaimedAmount);
+ _amount = unclaimedAmount;
}
```

### — RESOLUTION

Status: Partially resolved.

**Guardian:** If a claim lands prior to the clawback, that amount is still taken by the user.

LOW L-02

ACKNOWLEDGED

# Any address that has ever vested is permanently disqualified as a migration receiver because totalConvertedAmounts is never reset

CATEGORY	Logical Error
LOCATION	contracts/incentive/RatioVester.sol:isFreshForTransfer() contracts/incentive/RatioVester.sol:_settle() contracts/incentive/RatioVester.sol:_withdraw() contracts/incentive/RatioVester.sol:transferVestingState() contracts/staking/RewardRouterUtils.sol:validateReceiver()
REVIEW	Main Review

## DESCRIPTION

`isFreshForTransfer` performs 10 per-account checks before an address may receive a migration. A clean exit clears almost all of them (excluding the transferred variables and the `govCap`): `_withdraw` deletes `cumulativeClaimAmounts` and `claimedAmounts`, zeroes `balances` and `pairAmounts`, and jumps `trancheStartIndex` to the tranche length, and `_claim` zeroes `unpaidClaimAmounts`. `totalConvertedAmounts` is the exception. It is only ever incremented, in `_settle`, and the single decrement lives in `transferVestingState` on the sender side, in a branch that simultaneously sets `transferredCapDeductions[_sender]`, which is itself one of the freshness conditions. So no reachable exit returns a used address to fresh.

```

```solidity
// isFreshForTransfer requires ten fields to be zero, including:
&& totalConvertedAmounts[_account] == 0 // never reset by any exit path

// _settle only ever increments it
totalConvertedAmounts[_account] = totalConvertedAmounts[_account] + totalDelta;

// _withdraw clears the other counters but leaves totalConvertedAmounts untouched
delete cumulativeClaimAmounts[_account];
trancheStartIndex[_account] = tranches[_account].length;
```

```

Because `_settle` runs inside `_withdraw`, an ordinary deposit-then-withdraw is enough to trigger this, not only completed vesting. With the configured `vestingDuration` of 365 days, depositing `1e18` `esGMX` and withdrawing one second later still converts a nonzero amount, so `totalConvertedAmounts` becomes nonzero and no exit ever resets it. `RewardRouterUtils.validateReceiver` consumes this predicate for every designated vester, so `signalTransfer` and `acceptTransfer` revert with `StakingReceiverNotFresh`, and no user action or admin function can clear it.

**L -** Any address that has ever vested is permanently disqualified as a  
**02** migration receiver because `totalConvertedAmounts` is never reset

CONTINUED

The independent impact is narrow. `validateReceiver` also requires all six staking trackers to have `averageStakedAmounts == 0 && cumulativeRewards == 0`, and any account that has materialized rewards is already permanently non-fresh through those tracker checks, so the `totalConvertedAmounts` conjunct changes nothing for it. The conjunct is load-bearing only for the unmaterialized case: a first-time or passive staker whose tracker `cumulativeRewards` and `averageStakedAmounts` are still zero can deposit into and withdraw from a `RatioVester` (which moves only the `feeGmxTracker` receipt via a plain ERC20 transfer and never calls `_updateRewards`), leaving the trackers fresh but `totalConvertedAmounts` nonzero, so it is blocked solely by this conjunct.

#### — RECOMMENDATION

Dropping the `totalConvertedAmounts == 0` conjunct from `isFreshForTransfer` would allow the user to be a recipient again.

However, if that user's cap is below their current `totalConvertedAmounts`, a part of the transferred cap (up to 100%) can be consumed by that.

Choose the mechanism that you like best and stick with it!

#### — RESOLUTION

Status: Acknowledged.

LOW **L-03**

RESOLVED

# A frozen receiver passes `isFreshForTransfer`, so `signalTransfer` green-lights a migration that `acceptTransfer` always reverts after doing all its work

|          |                                                                                                                                                                        |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Logical Error                                                                                                                                                          |
| LOCATION | RatioVester.isFreshForTransfer / RatioVester.transferVestingState / RewardRouterV3.signalTransfer / RewardRouterV3.acceptTransfer / RewardRouterUtils.validateReceiver |
| REVIEW   | Main Review                                                                                                                                                            |

## — DESCRIPTION

`isFreshForTransfer` enumerates the ten per-account conditions a migration receiver must satisfy, but the frozen flag is not one of them. `INCENTIVE_GUARDIAN` can freeze any address with `setAccountFrozen`, including a brand-new all-zero address that still satisfies every freshness condition, which produces an account that is simultaneously fresh and frozen. Both router validation points read only `isFreshForTransfer`: `signalTransfer` validates the receiver and records the pending transfer, and `acceptTransfer` validates the receiver again before running the migration, so neither observes the freeze while `transferVestingState` itself rejects a frozen receiver.

```

``solidity
function isFreshForTransfer(address _account) public view override returns (bool) {
 return balances[_account] == 0
 && pairAmounts[_account] == 0
 && cumulativeClaimAmounts[_account] == 0
 && claimedAmounts[_account] == 0
 && unpaidClaimAmounts[_account] == 0
 && totalConvertedAmounts[_account] == 0
 && transferredCaps[_account] == 0
 && transferredCapDeductions[_account] == 0
 && govCapDeductions[_account] == 0
 && tranches[_account].length == trancheStartIndex[_account];
 // isFrozen[_account] is never checked, yet transferVestingState reverts on it
}
``

```

**L - 03** A frozen receiver passes `isFreshForTransfer`, so `signalTransfer` green-lights a migration that `acceptTransfer` always reverts after doing all its work **CONTINUED**

`signalTransfer` therefore succeeds and writes `pendingReceivers` for a frozen receiver, giving the sender a green light with no on-chain signal that the paired accept will fail. When the receiver calls `acceptTransfer`, validation passes, the router performs the whole migration ( `_compound`, the four staking moves, the pending-issuance claim, and the esGMX, bnGMX and GLP sweeps), and only the final designated-vestor loop reverts because `transferVestingState` rejects the frozen receiver.

#### — RECOMMENDATION

Add `!isFrozen[_account]` as a conjunct of `isFreshForTransfer`, or expose a single `canTransferVestingState(sender, receiver)` view that folds every `transferVestingState` revert condition, so the pre-flight predicate the router and any interface checks matches the predicate `acceptTransfer` actually enforces. If the frozen state is deliberately kept out of freshness, document that a caller validating a receiver must check both `isFreshForTransfer` and `isFrozen` before signalling, since the freshness view alone does not prove a migration will succeed.

#### — RESOLUTION

Status: Resolved.

## INFORMATIONAL I-01

ACKNOWLEDGED

# zeroCaps skip branch and withdrawToken emit no protocol event

|          |                                                                                                                                                                |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Events                                                                                                                                                         |
| LOCATION | contracts/incentive/VesterCapZeroer.sol:zeroCaps() / contracts/incentive/RatioVester.sol:withdrawToken() / contracts/incentive/EsGmxIssuer.sol:withdrawToken() |
| REVIEW   | Main Review                                                                                                                                                    |

## — DESCRIPTION

The incentive module routes almost every state change through IncentiveEventUtils and the EventEmitter, but two mutating paths that matter for off-chain monitoring emit nothing. `VesterCapZeroer.zeroCaps` emits `VesterCapZeroed` only on the branch that writes a deduction, so an account short-circuited by the `getMaxVestableAmount(account) == 0` guard produces no event at all. `withdrawToken` on both `RatioVester` and `EsGmxIssuer` moves protocol-held tokens out under TIMELOCK\_MULTISIG with no dedicated protocol event.

The `zeroCaps` skip branch is the material one: with no event, a silently-skipped account is indistinguishable in the log stream from an account that was never included in a batch, so a keeper or monitor cannot confirm from logs which accounts were actually closed. `withdrawToken` still triggers the underlying token's own ERC20 `Transfer` from the contract address, so the outflow stays traceable, but it is not tied to a protocol-level event the way every other value movement in the module is.

## — RECOMMENDATION

Emit a distinct event on the `zeroCaps` skip branch so a skipped account is distinguishable from an unprocessed one, and add a protocol-level event to both `withdrawToken` functions, so both paths match the module's own EventEmitter convention.

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-02

RESOLVED

# epochs[i].totalAmount and totalIssuedAmount diverge after a reduction and the difference is undocumented

CATEGORY Documentation

LOCATION contracts/incentive/EsGmxIssuer.sol:distributeEpoch() contracts/incentive/EsGmxIssuer.sol:reduceIssuedAmount() contracts/incentive/EsGmxIssuer.sol:finalizeEpoch()

REVIEW Main Review

## DESCRIPTION

distributeEpoch increases both `epochs[i].totalAmount` and `totalIssuedAmount` by the same batch total, but `reduceIssuedAmount` decreases only `totalIssuedAmount`. After any reduction the two diverge, so the sum of `epochs[i].totalAmount` over all epochs no longer equals `totalIssuedAmount`.

This is intended behavior, not an accounting error: `epochs[i].totalAmount` is the immutable per-epoch distributed record consumed only by `finalizeEpoch`'s reconciliation against the off-chain expected total, while `totalIssuedAmount` is the live net figure used for the backing checks in `distributeEpoch` and `withdrawToken`. No code sums the epochs or asserts the equality, so there is no functional impact. Neither field carries a comment stating this, so the divergence is easy to misread as a solvency mismatch.

## RECOMMENDATION

Document the differing semantics next to the declarations: state that `epochs[i].totalAmount` records the amount distributed in that epoch for reconciliation and is deliberately not adjusted by `reduceIssuedAmount`, whereas `totalIssuedAmount` is the live aggregate net of reductions used for solvency, so the sum of `epochs[i].totalAmount` is not expected to equal `totalIssuedAmount` after any reduction.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-03

RESOLVED

# Uncapped mode accepts conflicting issuer

CATEGORY Configuration

LOCATION <https://github.com/GuardianOrg/gmx-synthetics-private-team1-1784151983342/blob/84d06edb9ec28c73cfbddb1c2f634778dd9278a4/contracts/incentive/RatioVester.sol#L248-L253>

REVIEW Main Review

## DESCRIPTION

`RatioVester` permits deployment with both a nonzero `issuer` and `isUncapped` set to true. However, `getVestingCap()` only respects `isUncapped` when `issuer` is `address(0)`, silently treating the conflicting configuration as capped.

## RECOMMENDATION

Reject the conflicting configuration in `constructor()`.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-04

RESOLVED

# IRatioVester exposes no view for provisioningComplete or the transfer feature flag, so no complete migration pre-flight predicate can be built

CATEGORY Best Practices

LOCATION RatioVester.transferVestingState / RatioVester.provisioningComplete / IRatioVester / RewardRouterUtils.validateReceiver / RewardRouterV3.signalTransfer / RewardRouterV3.acceptTransfer

REVIEW Main Review

## DESCRIPTION

`transferVestingState` guards on seven conditions before it moves any state: the transfer feature flag, `provisioningComplete`, a self-transfer check, the sender frozen flag, the receiver frozen flag, `hasOpenSession(sender)`, and `isFreshForTransfer(receiver)`. `IRatioVester`, the type the router holds every designated vester as, exposes views for `isFrozen`, `hasOpenSession`, `isFreshForTransfer`, `getVestingCap`, `claimable`, `pairAmounts` and `issuer`, but it exposes no view for `provisioningComplete` or the transfer feature flag. `provisioningComplete` is a public variable on the concrete contract yet is absent from the interface, and the transfer feature flag lives in `DataStore` under `incentiveVesterTransferFeatureDisabledKey(address(this))` with no getter on the vester at all. `RewardRouterUtils.validateReceiver`, the only pre-flight the router runs, checks `isFreshForTransfer` and the tracker freshness conditions and consults neither of these two.

```
```solidity
```

```
// transferVestingState begins with two guards that have no IRatioVester view:
```

```
FeatureUtils.validateFeature(dataStore, Keys.incentiveVesterTransferFeatureDisabledKey(address(this)));
```

```
if (!provisioningComplete) { revert Errors.VesterProvisioningNotComplete(); }
```

```
// IRatioVester surfaces
```

```
isFrozen/hasOpenSession/isFreshForTransfer/getVestingCap/claimable/pairAmounts/issuer,
```

```
// but neither provisioningComplete nor any transfer-feature view
```

```
```
```

Because `signalTransfer` and `validateReceiver` never observe these two conditions, a sender can signal and a receiver can begin `acceptTransfer` against a vester whose provisioning is not complete or whose transfer feature is disabled, and the call reverts at the

## I - IRatioVester exposes no view for provisioningComplete or the transfer feature flag, so no complete migration pre-flight predicate can be built CONTINUED

designated-vestor loop with `VesterProvisioningNotComplete` or `DisabledFeature` only after the router has already run `_compound` and the staking moves. No complete migration pre-flight predicate can be assembled from the interface, so a successful `signalTransfer` does not imply the paired `acceptTransfer` will succeed.

### — RECOMMENDATION

Add `provisioningComplete` and a transfer-feature view to `IRatioVester`, or expose a single `canTransferVestingState(sender, receiver)` view that folds every `transferVestingState` revert condition, and have `signalTransfer` and `validateReceiver` consult it so the pre-flight predicate matches the execution predicate.

### — RESOLUTION

Status: Resolved.

## INFORMATIONAL I-05

RESOLVED

# signalTransfer accepts a self-transfer that transferVestingState always reverts

CATEGORY Out Of Scope  
LOCATION RewardRouterV3.signalTransfer  
REVIEW Main Review

## DESCRIPTION

`signalTransfer` records a pending migration without checking that the receiver differs from the caller, while `transferVestingState` rejects a self-transfer with `SelfVestingTransfer`. A fresh account satisfies both `_validateSender` and `_validateReceiver`, so calling `signalTransfer(msg.sender)` passes validation and writes `pendingReceivers[msg.sender] = msg.sender`, meaning the pre-flight the router exposes accepts a target that execution is guaranteed to reject.

```
```solidity
// RewardRouterV3.signalTransfer: no check that _receiver != msg.sender
_validateReceiver(_receiver);
pendingReceivers[msg.sender] = _receiver;

// RatioVester.transferVestingState: rejects a self-transfer
if (_sender == _receiver) { revert Errors.SelfVestingTransfer(_sender); }
```
```

Because `signalTransfer` succeeds, the caller receives a green light implying the migration is valid. The paired `acceptTransfer` then passes its own validation and runs the migration flow before the designated-vestor loop calls `transferVestingState(msg.sender, msg.sender)` and reverts `SelfVestingTransfer`, rolling the whole call back.

## RECOMMENDATION

Reject a self-target in `signalTransfer` by requiring `_receiver != msg.sender`.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-06

RESOLVED

# Ambiguous claimable() semantics during freeze

|          |                                                                                                                                                                                                                                                                                                                                                           |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | Unexpected Behavior                                                                                                                                                                                                                                                                                                                                       |
| LOCATION | <a href="https://github.com/GuardianOrg/gmx-synthetics-private-team1-1784151983342/blob/84d06edb9ec28c73cfbddb1c2f634778dd9278a4/contracts/incentive/RatioVester.sol#L301-L303">https://github.com/GuardianOrg/gmx-synthetics-private-team1-1784151983342/blob/84d06edb9ec28c73cfbddb1c2f634778dd9278a4/contracts/incentive/RatioVester.sol#L301-L303</a> |
| REVIEW   | Main Review                                                                                                                                                                                                                                                                                                                                               |

## DESCRIPTION

When an account is frozen, `claimable()` may return a positive amount derived from its already-settled `cumulativeClaimAmounts` and `unpaidClaimAmounts`, while `_claim()` unconditionally reverts with `VesterAccountFrozen`. Conversely, unsettled time-accrued conversion is omitted because `_pendingConversion()` returns zero for frozen accounts.

Also, if the account is unfrozen, the actual claimable amount may be more than what the `claimable()` function reported.

## RECOMMENDATION

Consider whether this is the right behavior for the function

## RESOLUTION

Status: Resolved.

INFORMATIONAL I-07

RESOLVED

# New cap retroactively unlocks old vesting

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/gmx-synthetics-private-team1-1784151983342/blob/84d06edb9ec28c73cfbddb1c2f634778dd9278a4/contracts/incentive/RatioVester.sol#L452-L459>

REVIEW Main Review

## DESCRIPTION

Cap-constrained tranches continue aging from their original `startTime`. For example, Alice can deposit 1,000 `esGMX`, have her cap reduced from 1,000 to 500, and claim 500 GMX after one year. If her cap is subsequently increased by 1,000 to support new rewards, the remaining 500 from her fully aged tranche becomes immediately claimable, leaving only 500 of the new headroom for fresh vesting.

The same behavior applies to frozen accounts. Deducting the account's cap before unfreezing prevents immediate settlement, but does not reset its tranche clock. A subsequent issuance or cap restoration creates headroom that can be consumed immediately by rewards that aged while the account was frozen.

## RECOMMENDATION

There are already tests that show the reported behavior, but consider whether the described scenario in the finding - where a new cap is added with the intention of vesting it over a year - is acceptable as well.

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-08

ACKNOWLEDGED

# Fully reversible deposits permit a renewable transient lock of all vesting backing

|          |                                                                                          |
|----------|------------------------------------------------------------------------------------------|
| CATEGORY | denial of service / MEV                                                                  |
| LOCATION | packages/gmx-synthetics-private/contracts/incentive/RatioVester.sol:_deposit():L327-L381 |
| REVIEW   | Main Review                                                                              |

## — DESCRIPTION

RatioVester reserves claimable-token backing for a deposit's full principal, but a same-timestamp withdrawal reverses that reservation without consuming cap or vesting. An eligible account can therefore front-run a competing deposit by reserving all backing, let the victim's deposit revert, and immediately withdraw to restore all balances; the same position can repeat around later transactions.

Impact: This enables renewable selective denial of public deposits for only gas and transient collateral use. Victims may miss a terminal vesting window if repeatedly censored, although retries, private ordering, and submitting when capacity is free mitigate practical impact. No insolvency or direct asset theft is demonstrated, so Low severity is appropriate.

## — RECOMMENDATION

Consider documenting this as a possible griefing scenario when the claimable token balance isn't high enough.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/SentryRepo-Incentivize-Contracts---Round-1-01a07bc4c3fa75a58f6034e9f4cbf817/blob/refinery/poc-01a08029-5151-7311-9262-a30cedc32167/test/refinery-poc/fully-reversible-deposits-permit-a-renewable-transient-lock--94842270.sol>

## — RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-09

RESOLVED

# Batch completion events do not index batchIndex

|          |                                                                                                                        |
|----------|------------------------------------------------------------------------------------------------------------------------|
| CATEGORY | logic                                                                                                                  |
| LOCATION | packages/gmx-synthetics-private/contracts/incentive/IncentiveEventUtils.sol:emit<br>IssuanceBatchDistributed():L35-L50 |
| REVIEW   | Main Review                                                                                                            |

## DESCRIPTION

Each successful distributeEpoch batch emits EsGmxIssuanceBatchDistributed via EventLog1 with only epochId as the indexed protocol topic, while batchIndex is stored only in dynamic event data. Multiple distinct batch indexes in the same epoch therefore produce identical indexed topics.

Impact: Consumers cannot efficiently filter logs by batchIndex at the RPC topic level and any noncanonical integration treating (event name, epoch topic) as a unique batch key could conflate batches. On-chain issuance, replay protection, claims, and canonical log identity (tx hash plus log index) remain unaffected, so impact is limited to monitoring/indexing usability.

## RECOMMENDATION

Emit completion using EventLog2 with epochId as topic1 and batchIndex as topic2, while retaining tx hash/log index as unique event identity.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/SentryRepo-Incentivize-Contracts---Round-1-01a07bc4c3fa75a58f6034e9f4cbf817/blob/refinery/poc-01a08029-5151-7311-9262-a30cedc32167/test/refinery-poc/batch-completion-events-do-not-index-batchindex-25080f43.sol>

## RESOLUTION

Status: Resolved.

## INFORMATIONAL I-01

ACKNOWLEDGED

# EsGmxIssuer header documentation omits public self-claim paths

|          |                                           |
|----------|-------------------------------------------|
| CATEGORY | Documentation                             |
| LOCATION | contracts/incentive/EsGmxIssuer.sol:18-23 |
| REVIEW   | Remediation Review                        |

## DESCRIPTION

The `EsGmxIssuer` header describes only controller-mediated claiming:

```
``solidity
// @dev Escrow ledger for weekly esGMX issuance. The off-chain engine distributes
// per-epoch batches through INCENTIVE_DISTRIBUTOR, INCENTIVE_ADMIN reconciles and
// seals each epoch, and the reward router (or a future claim wrapper) claims the
// issued esGMX for accounts through CONTROLLER. Issued amounts also back the bound
// RatioVester's vesting cap
``
```

The implementation also exposes public self-claim paths. Any account can call `claim()` or `claimAmount()` to claim its own issued esGMX. Only the functions that claim on behalf of another account are restricted to `CONTROLLER`:

```
```solidity
function claim() external override nonReentrant returns (uint256) {
    return _claim(msg.sender, claimable(msg.sender));
}

function claimAmount(uint256 _amount) external override nonReentrant returns (uint256) {
    return _claim(msg.sender, _amount);
}

function claimForAccount(address _account) external override nonReentrant onlyController returns (uint256) {
    {
        return _claim(_account, claimable(_account));
    }
}

function claimAmountForAccount(
    address _account,
    uint256 _amount
```

I-01 EsGmxIssuer header documentation omits public self-claim paths**CONTINUED**

```
) external override nonReentrant onlyController returns (uint256) {  
    return _claim(_account, _amount);  
}  
...
```

This was introduced within the reviewed range. Commit `1808e016` removed public self-claiming, gated claims through `CONTROLLER`, and updated the header to describe that design. Commit `69e1cbfc` subsequently restored `claim()` and `claimAmount()` but did not restore the header documentation.

This does not change access control or create an independent loss scenario. It can nevertheless mislead reviewers and integrators about the intended claim surface by implying that issued esGMX is claimed only through controller-authorized callers.

— RECOMMENDATION

Update the header to document both supported claim paths. For example:

```
``solidity  
// @dev Escrow ledger for weekly esGMX issuance. The off-chain engine distributes  
// per-epoch batches through INCENTIVE_DISTRIBUTOR, INCENTIVE_ADMIN reconciles and  
// seals each epoch, accounts claim their own issued esGMX through claim() or  
// claimAmount(), and CONTROLLER callers may claim on an account's behalf. Issued  
// amounts also back the bound RatioVester's vesting cap.  
``
```

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Misleading comment in reduceIssuedAmount ()

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/gmx-synthetics-private-team1-1784151983342/blob/0afb112f915f24de798d12782ce1f1e476c7d75/contracts/incentive/EsGmxIssuer.sol#L172-L176>

REVIEW Remediation Review

DESCRIPTION

The comment states that capping `_amount` prevents a prior claim from reverting the correction. However, if `claim()` fully consumes the claimable amount first, `unclaimedAmount` is zero, `_amount` is capped to zero, and `reduceIssuedAmount()` reverts with `EmptyIssuanceReduction`.

```
``solidity
// capped at the unclaimed amount so a claim landing first cannot make the correction revert
``
```

In addition, the `emitIssuanceReduced` event carries data only about the capped amount and not the requested.

RECOMMENDATION

Update the comment to document the full-claim behavior, or return early when `_amount` becomes zero if the correction should not revert. Also consider whether the current even emission is the intended one.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.